



“ Formando **nuevas generaciones** con **sello de excelencia** comprometidos con la **transformación social** de las regiones y un país en paz ”

Proyecto de autenticación multifactor

MFA

**Prototipo de acceso seguro para sistema de gestión
bibliotecaria mediante MFA**

Seguridad informática

05 de junio de 2026



1. Presentación del proyecto y formación de equipos

El trabajo final busca que fortalezcamos y apliquemos los conceptos de autenticación multifactor (MFA) mediante el desarrollo de un prototipo funcional o prueba de concepto en un entorno real o simulado. A través de este proyecto, nosotros los estudiantes comprenderán la importancia de la MFA como capa adicional de seguridad frente a ataques de robo de credenciales, suplantación o acceso no autorizado, y aplicarán los principios de verificación basada en “algo que sabes, algo que tienes y algo que eres”.

1.1 Nombre del proyecto

Prototipo de Acceso Seguro para Sistema de Gestión Bibliotecaria mediante Autenticación Multifactor (MFA).

1.2 Línea de Trabajo

Portal web con acceso seguro mediante MFA.

1.3 Equipo de trabajo y definición de roles

Las responsabilidades se han distribuido de manera equitativa para cubrir todas las fases del desarrollo, asegurando la colaboración en la toma de decisiones críticas.

Wilson Armando Rojas (Desarrollador Backend y Arquitecto de Base de Datos)

Responsabilidades: Diseño y creación del esquema en PostgreSQL. Desarrollo de la lógica del servidor utilizando Python. Gestión de sesiones y enrutamiento seguro de la aplicación.

Jeyson Varela (Ingeniero de Seguridad MFA y Frontend)

Responsabilidades: Implementación de la lógica de generación y validación de tokens OTP (One Time Password). Integración del envío de mensajes/correos. Diseño de las interfaces de usuario (HTML/CSS) y conexión de los formularios web con el backend.



2. Investigación del caso, justificación y diseño conceptual

2.1 Identificación del problema e investigación del caso

El entorno seleccionado es un Sistema de Gestión Bibliotecaria Universitario. Actualmente, los sistemas tradicionales basados únicamente en usuario y contraseña presentan vulnerabilidades críticas en dos escenarios principales dentro de este contexto:

- 2.1.1 Acceso a Bases de Datos Académicas Licenciadas:** Las universidades pagan altas sumas de dinero por licencias de plataformas como Scopus, IEEE y SpringerLink. Es una práctica común (y riesgosa) que los estudiantes compartan sus credenciales con personas externas a la institución, generando un uso no autorizado que viola los términos de las licencias y pone en riesgo la red universitaria.
- 2.1.2 Préstamos físicos y suplantación:** En el mostrador físico, un estudiante malintencionado o alguien que haya encontrado un carnet perdido podría solicitar libros a nombre de otro estudiante, generando pérdidas materiales y sanciones injustas para la víctima.

2.2 Justificación del uso de MFA y Mitigación de Riesgos

El entorno seleccionado es un Sistema de Gestión Bibliotecaria Universitario. Actualmente, los sistemas tradicionales basados únicamente en usuario y contraseña presentan graves vulnerabilidades, como el uso no autorizado de bases de datos académicas licenciadas debido al préstamo de credenciales entre estudiantes, lo cual tiene un impacto económico y legal para la institución.

Para mitigar estos riesgos, la implementación de MFA añade una capa estricta de validación, adaptada al nivel de privilegio de cada usuario sin romper la filosofía de confidencialidad de los factores (ningún código será verbalizado o compartido). La solución se divide en dos enfoques:

MFA para Personal Administrativo (Bibliotecario): Al tener acceso a todo el inventario y datos de estudiantes, su inicio de sesión requerirá credenciales + **TOTP** (Time-Based One Time Password), generado mediante una aplicación de autenticación (Google Authenticator), garantizando alta seguridad.

MFA para Estudiantes: Su inicio de sesión requerirá credenciales + **OTP por correo electrónico**, asegurando que solo el titular de la cuenta institucional pueda acceder a las bases de datos o gestionar sus préstamos.



Aprobación Segura de Préstamos: Para evitar la suplantación física en mostrador sin comprometer el OTP, el sistema funcionará mediante solicitudes asíncronas. El bibliotecario registrará el préstamo, pero este quedará en estado "pendiente". El estudiante, desde su propio dispositivo, deberá iniciar sesión en el *Portal Estudiante* (pasando por su proceso MFA) y confirmar la transacción directamente en su interfaz.

2.3 Borrador del Diseño Conceptual

El prototipo se dividirá en dos interfaces web diferenciadas bajo el mismo backend y base de datos, garantizando la separación de privilegios:

Portal Bibliotecario:

Autenticación: Login con MFA (Usuario + Contraseña + TOTP vía app móvil)

Funcionalidades: Gestionar inventario, registrar préstamos a estudiantes, y ver historial de estudiantes.

Portal Estudiante:

Autenticación: Login con MFA (Usuario + Contraseña + OTP por correo)

Funcionalidades: Ver préstamos activos, ver historial personal, y confirmar devolución.

Stack Tecnológico:

Backend: Python con Flask (enrutamiento, manejo de sesiones separadas por roles).

Base de Datos: PostgreSQL (almacenamiento de usuarios, roles, hashes de contraseñas y registro del estado de los préstamos).

Mecanismos MFA:

Librería PyOTP en Flask para generar los secretos TOTP del bibliotecario (validados contra apps como Google Authenticator).

Generador de tokens y módulo smtplib de Python para enviar los OTP temporales al correo del estudiante.

Flujo de Datos Seguro (Caso de Registro de Préstamo):

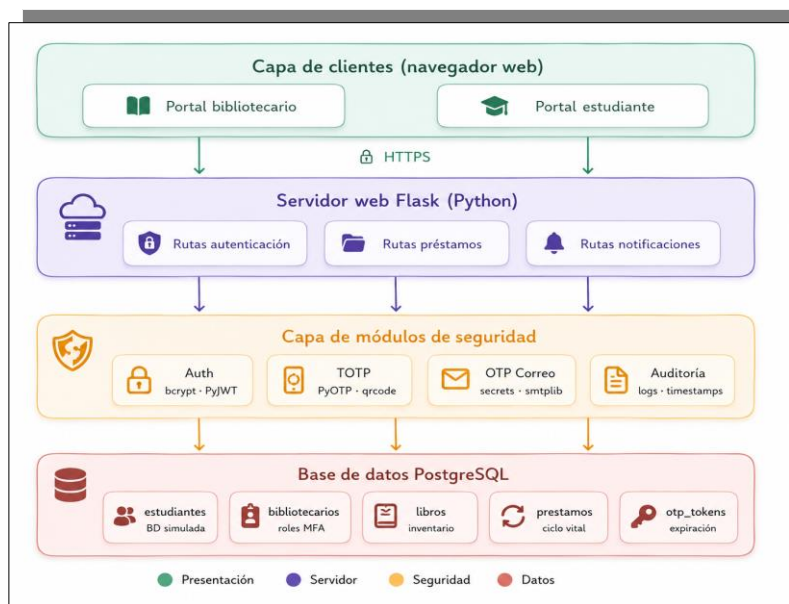


1. El Bibliotecario inicia sesión en su portal utilizando su usuario y el código TOTP de su app móvil.
2. En el módulo de "Registrar Préstamos", selecciona el libro y el estudiante, y emite la transacción.
3. El sistema guarda el préstamo en PostgreSQL con estado *Pendiente de Confirmación* y notifica al estudiante.
4. El estudiante abre el "Portal Estudiante" en su teléfono, ingresa sus credenciales y el código OTP que acaba de recibir en su correo institucional.
5. Una vez autenticado de forma segura en *su propia pantalla*, el estudiante visualiza el préstamo pendiente y hace clic en "Confirmar". La base de datos actualiza el estado a *Aprobado*.



Diseño de arquitectura del sistema

Visión General de la Arquitectura



Es el mapa estructural del sistema completo. Muestra cómo está organizado BiblioAuth en capas independientes que se comunican entre sí de forma ordenada y segura.

¿Por qué se organiza en capas? Porque separar responsabilidades es un principio fundamental de seguridad y buen diseño. Si una capa falla o necesita cambios, las demás no se ven afectadas. A esto se le llama bajo acoplamiento.

Sus cuatro capas y qué resuelve cada una

- **La capa de clientes** es todo lo que el usuario ve en el navegador. No contiene lógica de seguridad, solo presenta información y captura entradas. Está dividida en dos portales independientes porque cada rol tiene una superficie de acceso diferente. Que el estudiante no pueda navegar accidentalmente al panel del bibliotecario no es un detalle visual, es una decisión de seguridad.
- **La capa del servidor Flask** es el cerebro del sistema. Recibe cada petición HTTP, decide qué hacer con ella, llama a los módulos que necesita y devuelve una respuesta. Todo el tráfico entre el cliente y el servidor viaja por HTTPS, lo que garantiza que las contraseñas y los códigos OTP no viajen en texto plano por la red.
- **La capa de módulos de seguridad** es donde vive la lógica criptográfica real del proyecto. Es la capa más importante académicamente porque es donde se aplican los conceptos del curso. Aquí es donde bcrypt hasha las contraseñas, PyOTP genera y verifica los TOTP, secrets

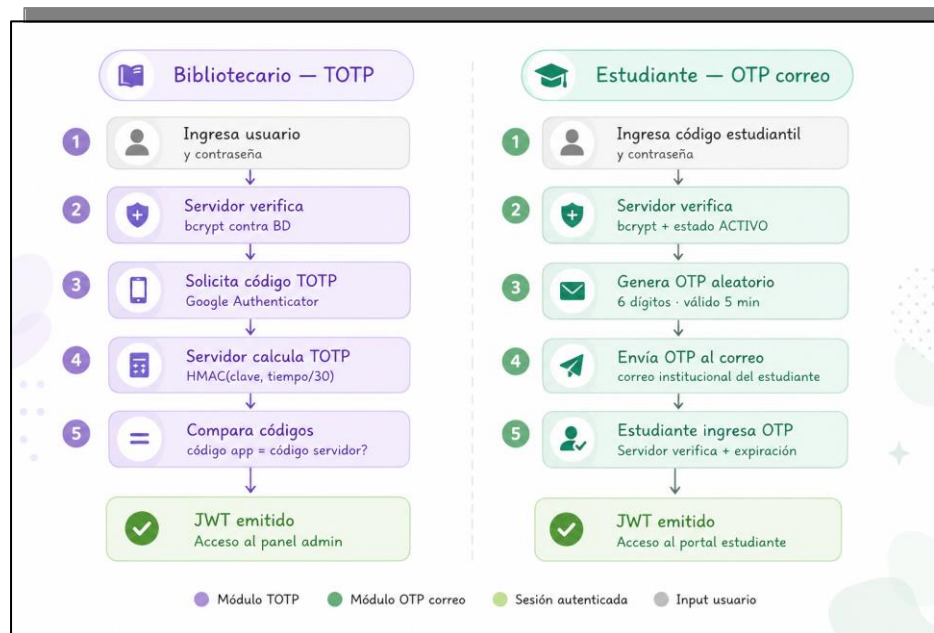


produce los OTP aleatorios para los estudiantes, y el módulo de auditoría registra cada operación con su timestamp.

- **Capa de base de datos PostgreSQL** es la memoria persistente del sistema. Guarda todo lo que el servidor necesita recordar entre una petición y la siguiente: usuarios, libros, préstamos activos y tokens OTP pendientes de verificación.

Flujo de Autenticación por Rol

El siguiente diagrama muestra cómo cada rol atraviesa su propio mecanismo MFA antes de acceder al sistema.



Este diagrama muestra el paso a paso cómo cada tipo de usuario atraviesa el proceso de verificación de identidad antes de acceder al sistema. No es un único flujo genérico sino dos flujos paralelos y distintos, uno para el bibliotecario y uno para el estudiante, cada uno con su propio mecanismo de segundo factor.

¿Por qué son flujos diferentes y no uno solo? Porque el nivel de confianza, la frecuencia de acceso y el contexto de cada rol son distintos. Aplicar el mismo mecanismo a ambos sería ignorar que la seguridad debe ser proporcional al riesgo y conveniente para el usuario legítimo.

- **El flujo del bibliotecario con TOTP** funciona así: primero verifica su contraseña, que el servidor compara contra la base de datos usando bcrypt. Si es correcta, el sistema le pide el código de seis dígitos que muestra Google Authenticator en ese momento. Lo importante aquí es entender que el servidor no le pregunta a la aplicación cuál es el código. Ambos lo calculan de forma independiente usando la misma clave secreta compartida durante el registro y el tiempo actual dividido en intervalos de 30 segundos. Si los dos resultados coinciden, la identidad está verificada. Esto es criptografía simétrica aplicada a autenticación en tiempo real.

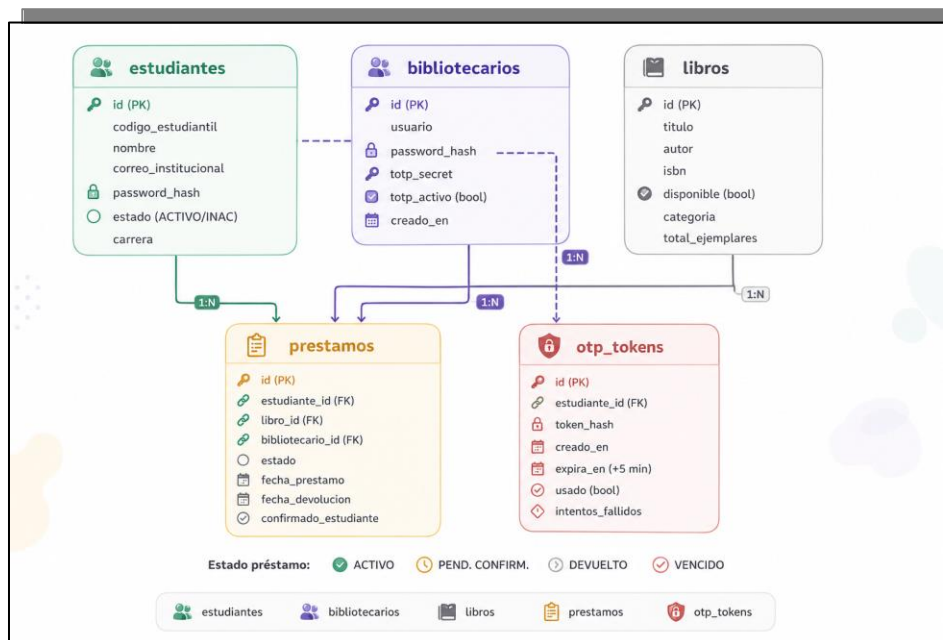


- **El flujo del estudiante con OTP por correo:** Sigue una lógica diferente. Después de verificar la contraseña, el servidor hace algo adicional que el flujo del bibliotecario no hace: comprueba que el estado del estudiante en la BD simulada sea ACTIVO. Un estudiante con estado INACTIVO no puede continuar aunque su contraseña sea correcta. Si pasa esa validación, el servidor genera un número aleatorio de seis dígitos usando el módulo secrets de Python, lo almacena en la tabla otp_tokens con una marca de expiración de cinco minutos, y lo envía al correo institucional del estudiante. El correo institucional aquí cumple doble función: es el canal de entrega del segundo factor y al mismo tiempo es la evidencia de que el estudiante pertenece a la institución.
- **La diferencia crítica entre los dos mecanismos:** Está en dónde vive el segundo factor. En TOTP el código se genera localmente en el dispositivo del bibliotecario sin necesidad de red, lo que lo hace más robusto ante fallos de conectividad y más resistente a interceptación. En OTP por correo el código viaja por la red hacia el estudiante, lo que introduce una dependencia de conectividad pero a cambio elimina la necesidad de que el estudiante instale cualquier aplicación adicional.

Ambos flujos terminan exactamente igual, con la emisión de un JWT firmado por el servidor. A partir de ese punto el sistema sabe quién es el usuario, cuándo se autenticó y cuánto tiempo tiene válida su sesión, sin necesidad de consultar la base de datos en cada petición.

Modelo de Base de Datos

Las cinco tablas del sistema y sus relaciones clave:





Las relaciones entre tablas siguen una lógica clara: un estudiante puede tener muchos préstamos activos e históricos, pero cada préstamo pertenece a exactamente un estudiante, un libro y un bibliotecario.

Descripción de cada una de las tablas:

- **estudiantes:** Es la BD simulada de la universidad. No la llena el sistema de biblioteca sino que viene precargada como si fuera una consulta al sistema académico institucional. El campo más relevante para seguridad es estado, porque es la barrera de autorización que separa a los estudiantes activos de los que ya no pertenecen a la institución.
- **bibliotecarios:** Guarda a los usuarios administradores del sistema. Su campo más crítico es totp_secret, que contiene la clave secreta compartida con Google Authenticator durante el registro. Esta clave debe tratarse con el mismo cuidado que una clave privada criptográfica porque quien la conozca puede generar códigos TOTP válidos sin tener el dispositivo físico del bibliotecario.
- **libros:** Es el inventario de la biblioteca. El campo disponible se actualiza automáticamente cada vez que se registra un préstamo o una devolución, garantizando que el sistema nunca permita prestar un libro que ya está fuera.
- **prestamos:** Es el corazón del sistema y también donde se materializa el concepto de no repudio. Guarda referencias tanto al estudiante como al bibliotecario que participaron en cada operación, con sus respectivos timestamps. El campo confirmado_estudiante es el que diferencia este sistema de una biblioteca tradicional: un préstamo no está completamente cerrado hasta que el estudiante confirma la devolución desde su portal autenticado con MFA.
- **otp_tokens:** Es temporal por naturaleza. Guarda los códigos OTP generados para los estudiantes junto con su timestamp de creación y su tiempo de expiración. El campo usado garantiza que un mismo código no pueda utilizarse dos veces aunque no haya expirado aún, lo que previene ataques de repetición. El campo intentos_fallidos permite bloquear la cuenta temporalmente si alguien intenta adivinar el código por fuerza bruta.



Definición de Herramientas

Esta es la stack tecnológica completa del proyecto con la justificación de cada elección:

Componente	Herramienta	Versión sugerida	Justificación
Lenguaje base	Python	3.11+	Nativo en Arch, ecosistema robusto para seguridad
Framework web	Flask	3.x	Ligero, sin magia oculta, ideal para demostrar flujos
MFA bibliotecario	PyOTP	2.9	Implementa RFC 6238 (TOTP) de forma transparente
Generación QR	qrcode	7.4	Genera el QR para registrar TOTP en Google Authenticator
MFA estudiante	secrets (stdlib)	—	Generación criptográficamente segura de OTP aleatorio
Envío de correo	smtplib (stdlib)	—	Envía el OTP al correo institucional del estudiante
Hash contraseñas	bcrypt	4.x	Estándar de la industria, sal automática, costo configurable
Tokens de sesión	PyJWT	2.8	JWT firmado para mantener la sesión post-autenticación
Base de datos	PostgreSQL	16	Sin servidor, perfecto para prototipo académico.
ORM opcional	SQLAlchemy	2.x	Abstrae las queries, previene SQL injection por diseño
Control versiones	Git + GitHub	—	Requerido por los entregables del proyecto
Cliente MFA	Google Authenticator	—	App gratuita, estándar de facto para TOTP
Entorno virtual	venv (stdlib)	—	Aisla dependencias, buena práctica desde el día uno



SC-CER96940

"Formando nuevas generaciones con sello de excelencia comprometidos con la transformación social de las regiones y un país en paz"
 Universidad de Pamplona
 Pamplona - Norte de Santander - Colombia
 Tels: (+57) 3153429495 - 3160244475
 www.unipamplona.edu.co